To refresh the token using Djoser, we make request to: *POST* ➔ *auth/jwt/refresh*

****

To Add new action, in *views.py*:

```
from rest_framework.decorators import action


class CustomerViewSet(CreateModelMixin, UpdateModelMixin, RetrieveModelMixin,
GenericViewSet):
    queryset = Customer.objects.all()
    serializer_class = CustomerSerializer

    @action(detail=False, methods=['GET', 'PUT'])
    def me(self, request):
        customer = Customer.objects.select_related('user').get(user_id =
request.user.id)
        if request.method == 'GET':
            serializer = CustomerSerializer(customer)
            return Response(serializer.data)
        elif request.method == 'PUT':
            serializer = CustomerSerializer(customer, data=request.data)
            serializer.is_valid(raise_exception=True)
            serializer.save()
            return Response(serializer.data)
```

For *serializers.py*:

Note: we must set user as *read_only* because nested object update not supported by default

```
class CustomerSerializer(serializers.ModelSerializer):
    user_id = serializers.IntegerField(read_only=True)
    user = UserSerializer(read_only=True)
    class Meta:
        model = Customer
        fields = ['id', 'user_id', 'phone', 'birth_date', 'membership', 'user']
```

****

For adding extra action to any class:

Note: If we set *detail to True* then it will apply for child routes like: *store/customer/{customerId}/me*

```
from rest_framework.decorators import action
class CustomerViewSet(CreateModelMixin, UpdateModelMixin, RetrieveModelMixin,
GenericViewSet):
    queryset = Customer.objects.all()
    serializer_class = CustomerSerializer

    @action(detail=False, methods=['GET', 'PUT'])
    def me(self, request):
        customer = Customer.objects.select_related('user').get(user_id =
request.user.id)
        if request.method == 'GET':
            serializer = CustomerSerializer(customer)
            return Response(serializer.data)
        elif request.method == 'PUT':
            serializer = CustomerSerializer(customer, data=request.data)
            serializer.is_valid(raise_exception=True)
            serializer.save()
            return Response(serializer.data)
             ************************************************************
```

To override the current user API of Djoser:

```
DJOSER = {
    'TOKEN_MODEL': None,
    'SERIALIZERS': {
        'user_create': 'core.serializers.UserCreateSerializer',
        'current_user': 'core.serializers.UserSerializer'
    },
    # other settings
}
            ---------------------------------------------------------------------------
```

```
from djoser.serializers import UserCreateSerializer as Base, UserSerializer as
Base02

class UserCreateSerializer(Base):

    class Meta(Base.Meta):
        fields = ['id', 'username', 'email', 'password', 'first_name',
'last_name']

class UserSerializer(Base02):
    class Meta(Base02.Meta):
        fields = ['id', 'first_name', 'email', 'last_name', 'username']
            ********************************************************
```

To handle permissions:

```
from rest_framework.permissions import IsAuthenticated, AllowAny


class CustomerViewSet(CreateModelMixin, UpdateModelMixin, RetrieveModelMixin,
GenericViewSet):
    queryset = Customer.objects.all()
    serializer_class = CustomerSerializer
    permission_classes = [IsAuthenticated]
    # here we return Objects not classes
    def get_permissions(self):
        if self.request.method == 'POST':
            return [AllowAny()]
        return [IsAuthenticated()]

    # Here also we can add permission_classes to action-decorator as list
    @action(detail=False, methods=['GET', 'PUT'])
    def me(self, request):
        customer = get_object_or_404(Customer.objects.select_related('user'),
user_id=request.user.id)
        if request.method == 'GET':
            serializer = CustomerSerializer(customer)
            return Response(serializer.data)
        elif request.method == 'PUT':
            serializer = CustomerSerializer(customer, data=request.data)
            serializer.is_valid(raise_exception=True)
            serializer.save()
            return Response(serializer.data)
            ********************************************************
```

To define the permissions classes:

```
from rest_framework import permissions

class IsAdminOrReadOnly(permissions.BasePermission):

    def has_permission(self, request, view): # type: ignore
        if request.method in permissions.SAFE_METHODS:
            return True
        return bool(request.user and request.user.is_staff)
```

To Define Model Permissions like customer services:

```
from rest_framework.permissions import IsAuthenticated, DjangoModelPermissions
class CustomerViewSet(ModelViewSet):
    queryset = Customer.objects.all()
    serializer_class = CustomerSerializer
    permission_classes = [DjangoModelPermissions]

    # here we return Objects not classes
    # def get_permissions(self):
    #     if self.request.method == 'GET':
    #         return [AllowAny()]
    #     return [IsAuthenticated()]

    # Here also we can add permission_classes to action-decorator as list
    @action(detail=False, methods=['GET', 'PUT'],
permission_classes=[IsAuthenticated])
    def me(self, request):
        customer = get_object_or_404(Customer.objects.select_related('user'),
user_id=request.user.id)
        if request.method == 'GET':
            serializer = CustomerSerializer(customer)
            return Response(serializer.data)
        elif request.method == 'PUT':
            serializer = CustomerSerializer(customer, data=request.data)
            serializer.is_valid(raise_exception=True)
            serializer.save()
            return Response(serializer.data)
```

The Implementation of *DjangoModelPermissions* can be mapped to *auth_permission-table*:

Note: when we implement this permission, the user must be authenticated and have the right permissions

```
    perms_map = {
        'GET': [],
        'OPTIONS': [],
        'HEAD': [],
        'POST': ['%(app_label)s.add_%(model_name)s'],
        'PUT': ['%(app_label)s.change_%(model_name)s'],
        'PATCH': ['%(app_label)s.change_%(model_name)s'],
        'DELETE': ['%(app_label)s.delete_%(model_name)s'],
    }
```

****

To create custom model permission:

- We add permission to Meta-class of target model
- Then we create:

```
class CanViewHistory(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.has_perm('store.view_history')
```

****

To create simple serializer for custom case:

```
class CreateOrderSerializer(serializers.Serializer):
    cart_id = serializers.UUIDField()

    def save(self, **kwargs):
        print(self.validated_data['cart_id']) # type: ignore
```

****

To create queryset that getting the data:

```
class OrderViewSet(ModelViewSet):
    # queryset = Order.objects.prefetch_related('items').all()
    serializer_class = OrderSerializer
    permission_classes = [IsAuthenticated]
    def get_queryset(self): # type: ignore
        user = self.request.user
        if user.is_staff:
            return Order.objects.select_related('customer').prefetch_related(
                models.Prefetch('items',
queryset=OrderItem.objects.select_related('product'))
            )
        else:
            t1, _= Customer.objects.get_or_create(user_id = user.id) # type:
ignore

            return
Order.objects.filter(customer_id=t1.id).select_related('customer').prefetch_relat
ed( # type: ignore
                models.Prefetch('items',
queryset=OrderItem.objects.select_related('product'))
            )
             ************************************************************
```

Note (To remember): we can't access request data or its contents from serializer, so we use context

```
class OrderViewSet(ModelViewSet):
    # queryset = Order.objects.prefetch_related('items').all()
    permission_classes = [IsAuthenticated]

    def get_serializer_class(self, *args, **kwargs): # type: ignore
        if self.request.method == 'POST':
            return CreateOrderSerializer
        return OrderSerializer

    def get_serializer_context(self):
        return { 'user_id': self.request.user.id, 'is_staff':
self.request.user.is_staff } # type: ignore
             ************************************************************
```

To save collection of data in one statement: `OrderItem.objects.bulk_create(order_items) # where order items is list of OrderItems-Objects`

****

Note (To remember): in this way we can set the permissions and allowed methods for user or admin:

```
class OrderViewSet(ModelViewSet):
    # queryset = Order.objects.prefetch_related('items').all()
    # permission_classes = [IsAuthenticated]

    http_method_names = ["get", "post", "patch", "delete"]

    def get_permissions(self):
        if self.request.method in ["PATCH", "DELETE"]:
            return [IsAdminUser()]
        return [IsAuthenticated()]

    def get_serializer_class(self, *args, **kwargs):  # type: ignore
        if self.request.method == "POST":
            return CreateOrderSerializer
        elif self.request.method == "PATCH":
            return UpdateOrderSerializer
        return OrderSerializer
```

****

To create signals when every user in our app is created, we can use signals:

```
from django.dispatch import receiver
from django.db.models.signals import post_save
from django.conf import settings
from .models import Customer

# sender: who sent the event to create the new customer
# in our case it is core.models.User
@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_new_customer_for_registered_user(sender, **kwargs):
    if kwargs['created']:
        Customer.objects.create(user=kwargs['instance'])
```

Then we must register this signals in our app.py of the current app (Like: store-app):

```
from django.apps import AppConfig


class StoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'store'

    def ready(self) -> None:
        import store.signals
            ************************************************************
```

To create custom signals, first we create the signal that we want:

```
from django.dispatch import Signal

order_created = Signal()
            ------------------------------------------------------------------------------------
```

Then inside the operation that we want we fire, or send it to the other apps, or other depends

```
from store.signals import order_created
```

Then inside any method of serializer, extra action, ...etc, we use:

Note 1: here we use *send_robust* because we want all listeners to receive the action, if we use send then if any one *fail* then send *will not send it to others*

```
Note 2: here the order is optional and we can get it in receiver using
kwargs['order']

order_created.send_robust(self.__class__, order=order)
            ------------------------------------------------------------------------------------
```

Then we register the receiver in other apps (like core)

```
from django.dispatch import receiver
from store.signals import order_created

@receiver(order_created) # type: ignore
def order_created(sender, **kwargs):
    print(kwargs['order'])
```

Then we import the receiver inside the ready function

```
from django.apps import AppConfig

class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'

    def ready(self) -> None:
        import core.signals.handlers
```

****